

AI-Curated Search Engine

Python crawling, local AI classification, search indexing, and a reader-facing API

Project context: GDSC IIT Roorkee project for discovering thoughtful blogs and articles	Status: Implemented prototype
--	--------------------------------------

A focused discovery system that crawls seed websites, rejects obvious noise, uses a local Gemma model to classify and summarize candidate pages, stores accepted records in MongoDB, indexes them in OpenSearch, and serves ranked results through FastAPI and React.

Technology stack

Python	Scrapy	Ollama / Gemma 3 1B	MongoDB
OpenSearch	FastAPI	React	Tailwind CSS

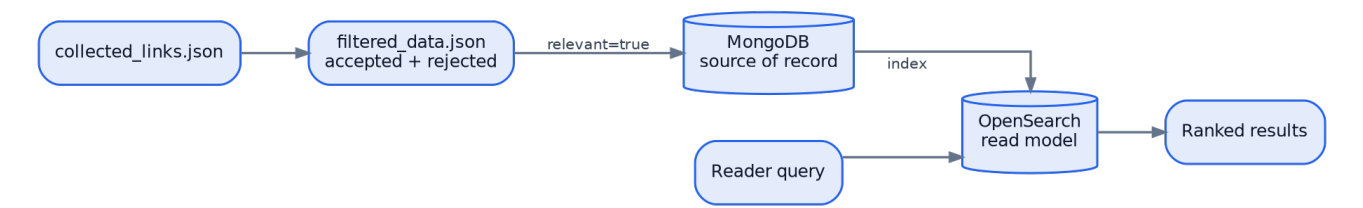
System at a glance



Curated collection Combines deterministic rejection rules with local-model relevance decisions.	Resumable processing Persists URL decisions so interrupted AI filtering can continue safely.
Read-optimized search Keeps MongoDB as curated record storage and OpenSearch as the query index.	Explainable results Returns a summary and selection reason alongside each original-source link.

System design and component ownership

The system deliberately separates an offline collection pipeline from the online search path. Expensive crawling and model inference occur before readers search, keeping query latency dependent mainly on OpenSearch and the API.



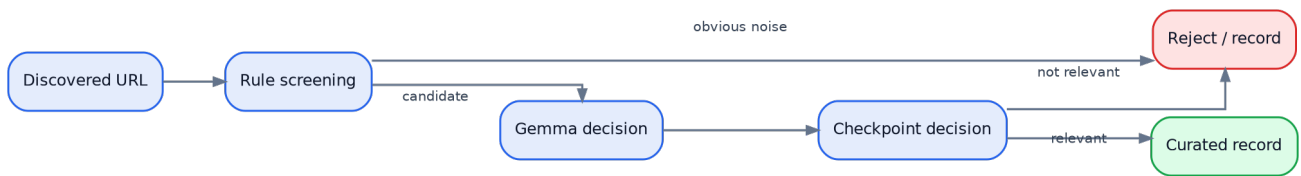
Component	Responsibility
Seed list and crawler	Discovers links from selected websites with a bounded crawl depth.
Fast screening	Rejects media, files, promotions, portals, and clearly unsuitable URLs.
Gemma AI filter	Classifies relevance and creates a concise summary and selection reason.
MongoDB importer	Stores accepted pages and skips URLs already present.
OpenSearch indexer	Copies curated fields into a search-optimized index.
FastAPI + React	Exposes search results and paginates them in the browser.

Key design decisions

- Use local inference for curation so page judgment is not coupled to a paid API.
- Keep collection as restartable scripts instead of blocking the reader-facing request path.
- Return links to original publishers rather than copying full article content.
- Weight title matches more heavily than summary and reason fields.

Core execution flow

A page moves through increasingly expensive filters. Cheap checks remove obvious noise first; only plausible articles are sent to the local model. Accepted records then become searchable through a separate indexing step.



Implementation notes

- Crawler output is stored as a URL list before AI processing.
- The model sees the title and the first portion of cleaned page text.
- Filtering saves after every URL and skips completed items on restart.
- The search endpoint performs a multi-field query and returns ranked JSON.
- The current frontend slices the returned result set into eight items per page.

Simplified curation loop

```
for url in discovered_urls:
    if already_processed(url):
        continue
    if fast_reject(url, html):
        save_decision(url, relevant=False)
        continue
    decision = gemma_classify(title, cleaned_text[:1200])
    save_decision(url, decision)
    if decision.relevant:
        store_curated_record(url, title, decision.summary, decision.reason)
```

This is a compact representation of the documented repository flow, not a verbatim source file.

Engineering review and production path

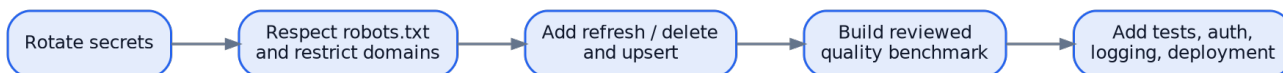
Engineering strengths

- Clear end-to-end data pipeline from discovery to search.
- Local AI curation and explainable selection fields.
- Separation of source-of-record storage from the read index.
- Restartable filtering reduces wasted model work.

Current limitations

- Repository documentation warns that credentials need rotation.
- Crawler currently does not obey robots.txt and can leave the seed domain.
- No reviewed benchmark or automated quality tests for AI decisions.
- API URL, CORS, and deployment assumptions are local-prototype oriented.

Recommended next steps



Why this is relevant to AI training work

- Demonstrates how to create difficult classification criteria rather than only call a model.
- Shows evaluation thinking through false accepts, false rejects, and human review.
- Requires reasoning about ranking, data quality, failure recovery, and edge cases.
- Communicates a multi-stage Python system in a form that can be independently reviewed.

Repository:

<https://github.com/SidheshwarSarangal/Search-Engin-New>

Scope and provenance:

Prepared from the repository README, architecture guide, storage/search guide, and documented current-boundary review. Implemented behavior and production recommendations are kept separate.

